



FIREBRAND

LYCEUM EDUCATION GROUP

Searching and Sorting Algorithms

C# Pathway | Session 6

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Lesson Aim and Objectives

Lesson Aim: Searching and Sorting Algorithms in C#

Lesson Objectives

Apply

- Apply searching and sorting algorithms in C#

Analyse

- Analyse algorithm behaviour and efficiency

Evaluate

- Evaluate algorithm suitability for different data scenarios

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

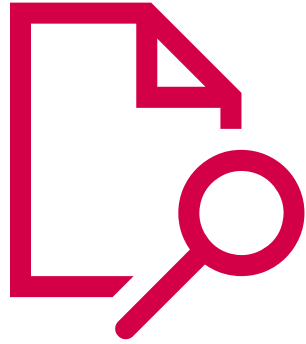
Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

Introduction to Search Algorithms



Linear Search



Binary Search

Linear Search

- **Sequential scan:** Checks every item from start to finish
- **Unsorted data:** Works perfectly fine on completely chaotic datasets
- **Example:** Looking for a specific book in an unorganised pile

Binary Search

- **Divide and conquer:** Repeatedly splits a dataset in half
- **Sorted requirement:** Only works if data is ordered sequentially
- **Example:** Finding a word in an alphabetical paper dictionary

Step-by-Step Search Mechanics

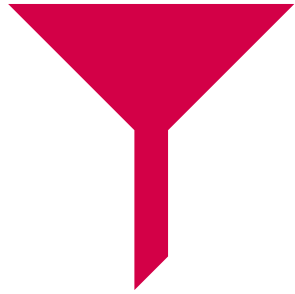
Linear Search Steps:

- Start at index 0 of the collection
- Compare the current item with the target value
- If it matches, return the index and terminate
- If not, move sequentially to the next slot

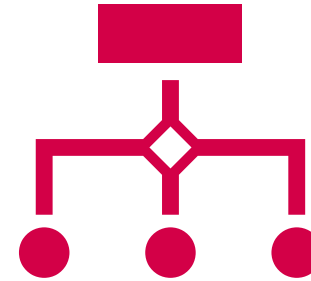
Binary Search Steps:

- Locate the middle element of the sorted list
- If the middle item matches the target, stop
- If the target is smaller, discard the right half
- If the target is larger, discard the left half
- Repeat the process on the remaining half

Introduction to Sorting



Bubble Sort
(Inefficient Baseline)



Efficient Sorting
(e.g., Merge Sort / Quick Sort)

Bubble Sort (Inefficient Baseline)

- **Neighbour comparison:** Compares adjacent elements side-by-side
- **Swapping mechanics:** Swaps pairs if they are in the wrong order
- **Bubbling effect:** Large values float to the end on each pass

Efficient Sorting (e.g. Merge Sort / Quick Sort)

- **Divide and conquer:** Splits lists into smaller sub-lists
- **Independent sorting:** Orders sub-units before combining them back
- **Scale performance:** Avoids scanning the entire collection repeatedly

Choosing the Right Tool for the Dataset

Algorithm	Data State	Optimal Dataset Size
Linear Search	Unsorted or Sorted	Small collections
Binary Search	Strictly Sorted	Large collections
Bubble Sort	Completely Unsorted	Tiny test arrays
Merge Sort	Completely Unsorted	Massive enterprise sets

Decision Rules:

- **Never use Binary Search** if your data cannot be sorted first
- **Avoid Bubble Sort** for anything larger than a few dozen items

Efficient vs. Inefficient Approaches

Scenario A:

E-Commerce ID Lookup

- **Context:** Finding a user profile in a database of 1,000,000 items
- **Inefficient:** Linear search takes up to 1,000,000 checks
- **Efficient:** Binary search finds it in under 20 checks

Scenario B:

Live Financial Transactions

- **Context:** Ordering 50,000 stock trades by timestamp per second
- **Inefficient:** Bubble sort takes 2.5 billion operations (freezes system)
- **Efficient:** Merge sort handles it easily in under 800,000 steps



FIREBRAND

LYCEUM EDUCATION GROUP

Physical Modelling Activity

Kinesthetic Sorting: Simulating Code with Objects

The Mission:

- **Objective:** Step outside the computer screen to simulate sorting algorithms manually using physical items
- **Materials:** A deck of numbered cards or physical items labelled with distinct values
- **Goal:** Observe how computers compare data values and swap memory slots under strict algorithm rules

The Constraints:

- **Hidden Values:** Cards must remain face down. You can only look at **two items at a time** during comparison phases
- **No Sneak Peeks:** You cannot scan the whole row at once to find the smallest number manually
- **Strict Memory Shifting:** Elements can only change positions by explicitly swapping adjacent positions

Executing a Bubble Sort Manually

Step-by-Step Instructions:

1. **Line Up:** Lay down a row of 6 unsorted cards face down
2. **Compare Pairs:** Flip over the first two cards on the left
3. **Evaluate and Swap:** If the left card is larger than the right card, swap their positions. Flip them back face down
4. **Shift Right:** Move one card to the right and repeat with the next adjacent pair
5. **Complete the Pass:** Continue down the line until you reach the end of the row. (The highest value is now locked at the end)
6. **Repeat:** Reset to the beginning of the row and repeat the loop until an entire pass happens with **zero swaps**

Reflection Checklist

Count the Operations

The Bubble Effect

Efficiency Realisation



FIREBRAND

LYCEUM EDUCATION GROUP

Guided Code Implementation

Building Search & Sort From Scratch



Objective:

- **Goal:** Build algorithms incrementally
- **Focus:** Write C# search engines
- **Focus:** Code basic sorting engines
- **Outcome:** Working, tracked implementations

The Roadmap:

- **Step 1:** Code linear search
- **Step 2:** Code bubble sort
- **Step 3:** Trace variable values
- **Step 4:** Modify array boundaries

Task 1: Linear Search

```
public static int LinearSearch(int[] arr, int target) {  
    for (int i = 0; i < arr.Length; i++) {  
        if (arr[i] == target) {  
            return i; } } // Match found  
    return -1; } // Match missing
```

Implementation Tasks:

- **Task a:**

Initialise integer data arrays

- **Task b:**

Code the sequential loop

- **Task c:**

Return correct element indexes

- **Task d:**

Handle missing search targets

Task 2: Bubble Sort

```
public static void BubbleSort(int[] arr) {  
    int n = arr.Length;  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                // Swap positions  
                int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1]  
= temp; } } } }
```

Implementation Tasks:

- **Task a:**

Create nested loop structures

Task b:

Set boundary restriction rules

Task c:

Code native C# swaps

Task d:

Return sorted data outputs

Tracing & Observation

Practical Code Review:

- **Trace:** Add diagnostic `Console.WriteLine()` statements
- **Trace:** Log every single swap
- **Trace:** Track loop variable changes
- **Observe:** Monitor shifting memory states

Verification Checks:

- **Test:** Confirm accurate target indexes
- **Test:** Check multi-pass sorted outputs
- **Test:** Validate structural code loops

Algorithm Walkthrough Challenge



ACTIVITY



Objective: Implement and compare search/sort algorithms on a provided datasets

Tasks / Requirements:

- Evaluate dataset scale and sorting states
- Choose optimal search and sort algorithms
- Defend selections using technical efficiency metrics

Key Point: Justify algorithm choice based on data size and whether it is already sorted

Extension Activity: Research and briefly explain how binary search compares in speed to linear search



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Comparative Evaluation Discussion



Algorithmic Performance and Scalability

The Evaluation Focus:

- **Performance metrics:** Measure algorithm execution and resource consumption
- **Real-world trade-offs:** Balance system speed against design complexity constraints
- **Core competency:** Explain architectural choice impacts on engineering scaling

Expected Outcome:

- **Defend selections:** Articulate why specific approaches fit concrete scenarios
- **Predict bottlenecks:** Spot processing failure points before deployment environments

Efficiency Considerations

Time Efficiency:

- **Operation counts:** Quantify steps executed as datasets scale up
- **Growth rates:** Map execution times mathematically using Big-O
- **Worst-case bounds:** Prepare system architectures for maximum structural load

Space Efficiency:

- **Memory footprint:** Track auxiliary RAM allocation during sorting actions
- **In-place updates:** Modify arrays directly to conserve system memory resources
- **Overhead constraints:** Avoid call-stack bloating during heavy processing runs

Practical System Trade-offs

The Development Trade-off Matrix:

- **Code complexity:** Advanced algorithms take significantly longer to write
- **Maintenance costs:** Highly optimised scripts can look unreadable during peer reviews
- **Data states:** Pre-sorting data takes time but opens fast search pathways

Real-World Infrastructure Realities:

- **Hardware limits:** Smart devices have tighter memory bounds than cloud servers
- **Network impacts:** Low latency requires swift data parsing algorithms

Scalability Case Analysis

Small Dataset Realities:

- **Negligible differences:** Primitive and advanced setups run almost identically
- **Simplicity wins:** Prioritise low development friction over complex indexing

Massive Scale Realities:

- **Total collapse:** Slow logic designs lock database threads entirely
- **Efficiency demands:** High-tier algorithms become absolute production requirements

Stretch and Challenge

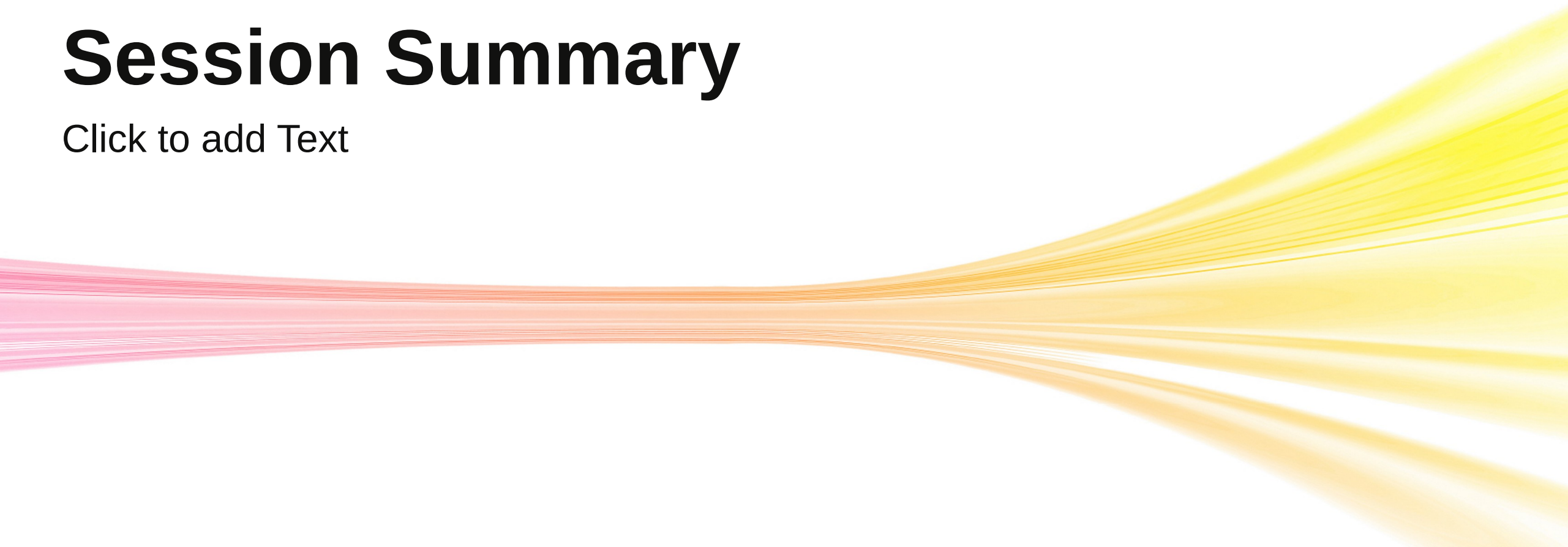
The Mission:

Upgrade your Linear and Bubble sort code into production-grade code

- **Task 1:** Refine code for clarity and readability
- **Task 2:** Reduce unnecessary operations to improve performance
- **Task 3:** Explain the impact of changes on efficiency and output

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Searching and Sorting Algorithms in C#

Lesson Objectives

Apply

- Apply searching and sorting algorithms in C#

Analyse

- Analyse algorithm behaviour and efficiency

Evaluate

- Evaluate algorithm suitability for different data scenarios

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

